

Xây dựng Agent đặt vé máy bay bằng LangChain

Họ và tên: Trần Thành Vinh

MSSV: 23521799

Môn học: SE373

1. Mục tiêu và công nghệ sử dụng

Bài tập xây dựng agent đặt vé máy bay, có khả năng tìm chuyến bay, kiểm tra ghế, đặt chỗ, thanh toán và xác minh kết quả. Agent phải tuân thủ ràng buộc của người dùng, kiểm tra quyền thực hiện hành động và bàn giao khi không thể tiếp tục.

Bài sử dụng LangChain để khai báo tool và triển khai agent ReAct qua `create_agent`, chạy trên nền LangGraph. Hai mẫu Plan-then-Execute và Lai được triển khai bằng bộ thực thi kế hoạch trong Python, sử dụng chung tool và harness để đối chiếu kết quả.

Model và dữ liệu chuyến bay được mô phỏng có kiểm soát, không cần API key và không thực hiện đặt vé hoặc thanh toán thật.

2. Tool mockup và các lớp harness

Hệ thống gồm năm tool:

- `search_flights`: tìm chuyến bay theo hành trình và ngày.
- `check_seat`: kiểm tra số ghế còn lại.
- `book_seat`: tạo mã đặt chỗ.
- `pay`: thanh toán đặt chỗ.
- `get_booking`: tra cứu trạng thái đặt chỗ.

Luồng xử lý chính: **Tìm chuyến** → **Kiểm tra ghế** → **Đặt chỗ** → **Thanh toán** → **Xác minh**.

Các lớp harness được cài đặt gồm:

Ràng buộc là dữ liệu:

Yêu cầu được lưu trong cấu trúc `RangBuoc`, gồm sân bay đi, sân bay đến, ngày bay, giờ giới hạn, giá tối đa và quyền đổi ngày. Kịch bản mặc định yêu cầu bay SGN → DAD ngày 07/10, trước 12:00, giá không quá 2.000.000 đồng. Harness kiểm tra ràng buộc trước khi đặt chỗ hoặc thanh toán.

Tiêu chí hoàn thành kiểm bằng code:

Agent chỉ được đánh giá thành công khi dữ liệu đặt chỗ có trạng thái `confirmed`, đã thanh toán và thỏa các ràng buộc về hành trình, ngày, giờ và giá. Kết luận của agent không được dùng thay cho việc kiểm tra dữ liệu thực tế.

Kiểm quyền:

Vé không hoàn tiền hoặc có giá vượt hạn mức tự quyết 1.500.000 đồng cần được duyệt trước khi đặt chỗ và thanh toán. Khi không được duyệt, agent dừng trước hành động bị chặn.

Bàn giao:

Khi cần người duyệt, phát hiện lặp hoặc kế hoạch không còn phù hợp, agent xuất lý do dừng, các hành động đã thử, trạng thái hiện tại và câu hỏi cho người dùng. Hệ thống có giới hạn số lượt gọi để tránh chạy vô hạn.

3. Ba mẫu thiết kế agent

- **ReAct:** lựa chọn hành động tiếp theo dựa trên kết quả tool vừa nhận được. Mẫu này có thể chuyển sang chuyển khác khi gặp lỗi hoặc hết ghế.
- **Plan-then-Execute:** lập kế hoạch trước rồi thực hiện lần lượt. Trong bài, mẫu này dừng và bàn giao khi kế hoạch trở nên lỗi thời.
- **Lai:** thực hiện theo kế hoạch, đồng thời theo dõi kết quả từng bước. Khi gặp thay đổi hoặc lỗi tool, agent lập lại kế hoạch để tiếp tục.

4. Kết quả đánh giá

Lệnh `python agent_dat_ve.py --danh-gia` thực hiện 13 lượt đánh giá. Mỗi lượt sử dụng kho dữ liệu mới; các yêu cầu duyệt trong bảng được mô phỏng là đồng ý.

Trong bảng, **M/T** là số lần gọi model và tool. Số lần gọi tool bao gồm bước xác minh hoàn thành.

Kịch bản	ReAct	Plan-then-Execute	Lai
s1 – Đặt vé thuận lợi	Đạt; M/T: 6/6	Đạt; M/T: 1/6	Đạt; M/T: 1/6
s2 – Chuyển rẻ nhất đã hết ghế	Đạt; M/T: 7/7	Bàn giao; M/T: 1/2	Đạt; M/T: 2/7
s3 – Phải chuyển sang ngày dự phòng	Đạt; M/T: 10/10	Bàn giao; M/T: 1/2	Đạt; M/T: 4/10
s4 – Model liên tục lặp hành động	Phát hiện lặp; M/T: 3/2	Không chạy	Không chạy
s5 – Ghế hết sau khi kiểm tra	Đạt; M/T: 8/8	Bàn giao; M/T: 1/3	Đạt; M/T: 2/8

Kịch bản s4 được thiết kế riêng để kiểm tra bộ phát hiện lặp của ReAct, nên không dùng để so sánh tỷ lệ thành công giữa ba mẫu.

Nhận xét: Plan-then-Execute dùng ít lần gọi model và phù hợp khi kế hoạch ổn định, nhưng không hoàn thành các tình huống cần thay đổi kế hoạch trong cách triển khai này. ReAct thích ứng được với kết quả tool nhưng dùng nhiều lần gọi model hơn. Mẫu Lai hoàn thành các tình huống thay đổi với số lần gọi model thấp hơn ReAct trong bộ mô phỏng.

Các trường hợp “không đạt” cần được phân biệt: agent chưa hoàn thành việc đặt vé, nhưng việc dừng đúng lúc để bàn giao vẫn là hành vi đúng của harness.

5. Minh chứng chạy chương trình

Hình 1 – Đánh giá ba mẫu thiết kế

```
python agent_dat_ve.py --danh-gia
| Kịch bản | Mẫu | Kết quả | Kiểu dừng | Gọi model | Gọi tool | Lập lại KH | Kiểm quyền |
|---|---|---|---|---|---|---|---|
| s1_thuan_loi | **react** | ✓ ĐẠT | ĐẠT (đích) | 6 | 6 | 0 | 0 |
| s1_thuan_loi | **plan** | ✓ ĐẠT | ĐẠT (đích) | 1 | 6 | 0 | 0 |
| s1_thuan_loi | **lai** | ✓ ĐẠT | ĐẠT (đích) | 1 | 6 | 0 | 0 |
| s2_rao_can | **react** | ✓ ĐẠT | ĐẠT (đích) | 7 | 7 | 0 | 2 |
| s2_rao_can | **plan** | ✗ KHÔNG ĐẠT | Kế hoạch lỗi thời → bàn giao | 1 | 2 | 0 | 0 |
| s2_rao_can | **lai** | ✓ ĐẠT | ĐẠT (đích) | 2 | 7 | 1 | 2 |
| s3_doi_huong | **react** | ✓ ĐẠT | ĐẠT (đích) | 10 | 10 | 0 | 0 |
| s3_doi_huong | **plan** | ✗ KHÔNG ĐẠT | Kế hoạch lỗi thời → bàn giao | 1 | 2 | 0 | 0 |
| s3_doi_huong | **lai** | ✓ ĐẠT | ĐẠT (đích) | 4 | 10 | 3 | 0 |
| s4_cung_dau | **react** | ✗ KHÔNG ĐẠT | Phát hiện lặp → bàn giao | 3 | 2 | 0 | 0 |
| s5_mat_ghze_sau_kiem | **react** | ✓ ĐẠT | ĐẠT (đích) | 8 | 8 | 0 | 2 |
| s5_mat_ghze_sau_kiem | **plan** | ✗ KHÔNG ĐẠT | Kế hoạch lỗi thời → bàn giao | 1 | 3 | 0 | 0 |
| s5_mat_ghze_sau_kiem | **lai** | ✓ ĐẠT | ĐẠT (đích) | 2 | 8 | 1 | 2 |
```

Đã lưu chi tiết từng lượt chạy vào: ket_qua_danh_gia.json

Bảng kết quả thể hiện 13 lượt chạy, trạng thái hoàn thành, số lần gọi model/tool, số lần lập lại kế hoạch và kiểm quyền.

Hình 2 – Mẫu Lai xử lý ghé hết sau khi kiểm tra

```
python agent_dat_ve.py --mau lai --kich-ban s5 --in-trace
=====
Kịch bản: s5_mat_ghze_sau_kiem - Ghế biến mất THẬT giữa chừng - thay đổi kho duy nhất trong bộ kịch bản: check_seat(VN214) còn 1 ghế, đến lượt đặt thì khách khác đã lấy (book_seat tr
ả flight_full) → agent phải chuyển sang ứng viên khác.
Mẫu : lai
=====
[KẾ HOẠCH #1] search_flights → check_seat → book_seat → pay → get_booking
[BƯỚC 1] search_flights({"di": "SGN", "den": "DAD", "ngay": "07/10"})
[BƯỚC 1] Tool {"status": "ok", "di": "SGN", "den": "DAD", "ngay": "07/10", "matched": 5, "flights": [{"so_hieu": "VN122", "gio": "08:00", "gia": 2310000, "hoan": false}, {"so_hie
u": "VJ604", "gio": "09:30", "gia": 1850000, "hoan": false}, {"so_hieu": "QH118", "gio": "10:15", "gia": 1980000, "hoan": true}, {"so_hieu": "VN214", "gio": "08:15", "gia": 1480000,
"hoan": true}, {"so_hieu": "VJ210", "gio": "11:40", "gia": 1690000, "hoan": false}]}
[BƯỚC 2] check_seat({"so_hieu": "VN214", "ngay": "07/10"})
[BƯỚC 2] Tool {"status": "ok", "so_hieu": "VN214", "ngay": "07/10", "con": 1, "gia": 1480000, "con_cho": true}
[BƯỚC 3] book_seat({"so_hieu": "VN214", "ngay": "07/10"})
[BƯỚC 3] Tool {"status": "error", "error": "flight_full", "so_hieu": "VN214", "ngay": "07/10", "hint": "Chọn chuyến khác còn ghế (check_seat để biết)"}
[XÉT LAI] TOOL_LOI - book_seat trả flight_full → lập lại kế hoạch
[KẾ HOẠCH #2] check_seat → book_seat → pay → get_booking
[BƯỚC 4] check_seat({"so_hieu": "VJ210", "ngay": "07/10"})
[BƯỚC 4] Tool {"status": "ok", "so_hieu": "VJ210", "ngay": "07/10", "con": 6, "gia": 1690000, "con_cho": true}
[BƯỚC 5] book_seat({"so_hieu": "VJ210", "ngay": "07/10"})
[BƯỚC 5] Tool {"status": "ok", "ma_dat": "4XJ1", "so_hieu": "VJ210", "gia": 1690000, "hoan": false, "trang_thai": "held", "paid": false, "ngay": "07/10", "gio": "11:40", "di": "S
GN", "den": "DAD"}
[BƯỚC 6] pay({"ma_dat": "4XJ1", "the": "corp_card"})
[BƯỚC 6] Tool {"status": "ok", "ma_dat": "4XJ1", "trang_thai": "paid", "so_tien": 1690000}
[BƯỚC 7] get_booking({"ma_dat": "4XJ1"})
[BƯỚC 7] Tool {"status": "ok", "ma_dat": "4XJ1", "so_hieu": "VJ210", "gia": 1690000, "hoan": false, "trang_thai": "confirmed", "paid": true, "ngay": "07/10", "gio": "11:40", "di":
": "SGN", "den": "DAD"}
=====
Kết quả : ✓ ĐẠT tiêu chí hoàn thành (kiểm bằng code)
Kiểu dừng : ĐẠT (đích)
Chi phí : 2 lần gọi model - 8 lần gọi tool - lập lại kế hoạch 1 lần
Kiểm quyền : book_seat({"so_hieu": "VJ210", "ngay": "07/10"}) - vé VJ210 KHÔNG hoàn, giá 1.690.000đ - vượt thẩm quyền tự quyết (hạn mức 1.500.000đ, vé không hoàn) → ĐỪNG Ý
Kiểm quyền : pay({"ma_dat": "4XJ1", "the": "corp_card"}) - vé VJ210 KHÔNG hoàn, giá 1.690.000đ - vượt thẩm quyền tự quyết (hạn mức 1.500.000đ, vé không hoàn) → ĐỪNG Ý
Ràng buộc : ĐẠT (không vi phạm ràng buộc dữ liệu)
Câu trả lời:
Đã đặt vé VJ210 mã 4XJ1, trạng thái confirmed, giá 1.690.000đ - thoả ràng buộc (SGN→DAD, trước 12:00, trăn 2.000.000đ); chuyển bay ngày 07/10 lúc 11:40.
```

Ban đầu VN214 còn một ghế, nhưng khi đặt chỗ tool trả lỗi flight_full. Agent lập lại kế hoạch, chọn VJ210 và hoàn thành đặt vé sau khi được duyệt. Kết quả dùng hai lần gọi model, tám lần gọi tool và một lần lập lại kế hoạch.

Hình 3 – Kiểm quyền và bàn giao

```
python agent_dat_ve.py --mau react --kich-ban s2 --khong-duyet
=====
Kịch bản: s2_rao_can - VN214 (rẻ nhất thoả ràng buộc) ĐÃ HẾT GHẾ từ trước (mock: bất khả dụng tính) → phải chuyển sang VJ210 1.690.000đ KHÔNG hoàn → Kịch hoạt lớp kiểm quyền (S41).
Mẫu : react
=====
Kết quả : ✗ KHÔNG đạt
Kiểu dừng : Cần người duyệt → bàn giao
Chi phí : 4 lần gọi model - 3 lần gọi tool
Kiểm quyền : book_seat({"so_hieu": "VJ210", "ngay": "07/10"}) - vé VJ210 KHÔNG hoàn, giá 1.690.000đ - vượt thẩm quyền tự quyết (hạn mức 1.500.000đ, vé không hoàn) → DỪNG chờ người du
yết
Câu trả lời:
DỪNG BẮT THƯỜNG - CẦN NGƯỜI DUYỆT - vé VJ210 KHÔNG hoàn, giá 1.690.000đ - vượt thẩm quyền tự quyết (hạn mức 1.500.000đ, vé không hoàn)
Đã thử : search_flights({"di": "SGN", "den": "DAD", "ngay": "07/10"}) → check_seat({"so_hieu": "VN214", "ngay": "07/10"}) → check_seat({"so_hieu": "VJ210", "ngay": "07/1
0"})
Trạng thái : {"ma_dat": None, "trang_thai": None, "paid": False, "dat_cho": [], "hanh_dong_bi_chan": "book_seat({"so_hieu": "VJ210", "ngay": "07/10"})", "so_lan_goi_tool": 3
, "so_lan_goi_model": 4}
Hỏi người dùng : Cho phép thực hiện hành động này không?
```

Agent bị chặn trước khi đặt vé VJ210 vì vé không hoàn tiền và vượt hạn mức tự quyết. Kết quả bàn giao cho người dùng; chưa tạo đặt chỗ và chưa thanh toán.

Hình 4 – Phát hiện lặp

```
python agent_dat_ve.py --mau react --kich-ban s4 --in-trace
=====
Kịch bản: s4_cung_dau - Model 'cứng đầu': mỗi lượt vẫn gọi check_seat('VJ604') bất kể quan sát → bộ phát hiện lặp phải báo động và bàn giao (S39, S46).
Mẫu : react
=====
[V1] AI check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'})
[V1] Tool {"status": "ok", "so_hieu": "VJ604", "ngay": "07/10", "con": 2, "gia": 1850000, "con_cho": true}
[V2] AI check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'})
[V2] Tool {"status": "ok", "so_hieu": "VJ604", "ngay": "07/10", "con": 2, "gia": 1850000, "con_cho": true}
[V3] AI check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'})
[V3] AI DỪNG BẤT THƯỜNG · LOOP · 'check_seat' gọi 3 lần với cùng tham số trong 6 vòng gần nhất
[V3] AI Đã thử : check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'}) → check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'}) → check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'})
[V3] AI Trạng thái : {'so_quan_sat_ok': 2}
[V3] AI Hỏi người dùng : Agent đang lặp vô ích – tiếp tục, đổi chiến lược hay dừng?
=====
Kết quả : ❌ KHÔNG đạt
Kiểu dừng : Phát hiện lặp → bàn giao
Chi phí : 3 lần gọi model · 2 lần gọi tool
Câu trả lời:
DỪNG BẤT THƯỜNG · LOOP · 'check_seat' gọi 3 lần với cùng tham số trong 6 vòng gần nhất
Đã thử : check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'}) → check_seat({'so_hieu': 'VJ604', 'ngay': '07/10'})
Trạng thái : {'so_quan_sat_ok': 2, 'ma_dat': None, 'trang_thai': None, 'paid': False, 'dat_cho': [], 'so_lan_goi_tool': 2, 'so_lan_goi_model': 3}
Hỏi người dùng : Agent đang lặp vô ích – tiếp tục, đổi chiến lược hay dừng?
```

Model đề xuất cùng một hành động kiểm tra ghế ba lần. Harness chặn lần thứ ba, nên thực tế chỉ có hai lần gọi tool, sau đó dừng và xuất thông tin bàn giao.

6. Kết luận

Bài triển khai năm tool mockup, bốn lớp harness và ba mẫu thiết kế agent. Kết quả cho thấy mẫu Lai kết hợp được khả năng thực hiện theo kế hoạch và thích ứng khi môi trường thay đổi.

Đánh giá sử dụng model mô phỏng nên số lần gọi chỉ phản ánh cách hoạt động của ba mẫu trong các kịch bản đã xây dựng; chưa đo chi phí token, độ trễ hoặc chất lượng của LLM thực tế.